Proyecto Administración de Bases de Datos
Proyecto Restaurante – FastAPI

Facultad de Ingeniería

Juan Felipe Galvis Vargas
Andrés David Ortiz
María Paula Rodríguez Mancera

Profesora: Ricardo Naranjo Faccini

Abril de 2026

Descripción del framework

Para el desarrollo de este proyecto se seleccionó FastAPI como framework principal. En términos generales, se trata de un framework de backend orientado a la construcción de APIs, desarrollado en Python, lo que ya de entrada lo vuelve una opción bastante atractiva para un equipo que busca rapidez de desarrollo, claridad en la lógica y, también hay que decirlo, una curva de trabajo razonable sin tener que entrar en estructuras excesivamente pesadas.

A diferencia de otros enfoques más tradicionales o más "cerrados", FastAPI se presenta como una alternativa moderna para construir la parte del servidor de una aplicación web. Es decir, no está pensado tanto como un framework monolítico que lo haga absolutamente todo, sino como una base fuerte para implementar la lógica del sistema, exponer servicios y gestionar la comunicación entre la aplicación y la base de datos. En este proyecto eso encaja bien porque el problema del restaurante exige precisamente eso: manejar autenticación, control de acceso por roles, persistencia de información y operaciones que deben responder de forma clara y consistente.

Según las características consideradas por el grupo, FastAPI utiliza Python como lenguaje de desarrollo y se clasifica como un framework de tipo Backend API. Además, puede ejecutarse en distintos sistemas operativos, entre ellos Linux, Windows y macOS, lo que aporta flexibilidad al momento de desarrollar, probar o desplegar la aplicación en diferentes entornos. En cuanto a compatibilidad con bases de datos, se contempló su uso junto con PostgreSQL, un motor ampliamente utilizado y adecuado para una aplicación transaccional como la que requiere la gestión de un restaurante.

Uno de los rasgos que más peso tuvo en su selección fue su enfoque en el rendimiento. FastAPI se describe como un framework muy rápido y basado en async, lo cual resulta especialmente útil en escenarios donde varias operaciones pueden ocurrir casi al mismo tiempo: registrar pedidos, actualizar estados, consultar disponibilidad de mesas o validar reservaciones sin cruces de horario. No significa que por usarlo todo se vuelva automático o perfecto, pero sí ofrece una base técnica que favorece respuestas ágiles y una organización más eficiente del backend.

Otro aspecto relevante es el nivel de complejidad que se le atribuye: intermedio. Esto sugiere que FastAPI no es una herramienta tan elemental como para limitar demasiado el diseño, pero tampoco resulta tan compleja como para volver inviable su adopción en un proyecto académico de este tipo. En otras palabras, exige cierto orden, comprensión del flujo backend y manejo adecuado de Python.

Desde el punto de vista de licenciamiento, FastAPI cuenta con licencia MIT, es decir, una licencia libre. Esto representa una ventaja importante, porque permite trabajar con el framework sin costos de uso y sin restricciones problemáticas para fines académicos.

Llevado al contexto concreto del proyecto, FastAPI resulta apropiado porque permite estructurar el sistema del restaurante desde una lógica de servicios backend bien definida. Esto favorece la implementación de procesos como la autenticación de usuarios, el control de acceso según rol, la gestión de reservaciones, el registro de pedidos, la actualización de estados y la persistencia de

datos. Además, su naturaleza como framework de backend hace que el diseño de la solución pueda enfocarse en separar con claridad la lógica del negocio, el acceso a datos y la interacción con la interfaz web.

Para resumir, FastAPI fue seleccionado por combinar varios elementos que, puestos juntos, hacen sentido para el proyecto: uso de Python, compatibilidad con bases de datos adecuadas, velocidad, soporte para programación asíncrona, portabilidad entre sistemas operativos y un nivel técnico intermedio que permite construir una solución robusta sin apartarse del alcance académico del trabajo.

Arquitectura Utilizada

La arquitectura propuesta para el sistema de gestión del restaurante parte de una idea bastante clara: construir una solución web que sea ordenada, coherente con FastAPI y, al mismo tiempo, suficientemente simple como para poder implementarse dentro del alcance del proyecto. No se buscó una arquitectura innecesariamente compleja ni una distribución exagerada en múltiples servicios. En cambio, se optó por una arquitectura cliente-servidor, organizada como un monolito modular, en donde la interfaz web, la lógica del negocio y la persistencia de datos se encuentran separadas de forma lógica, aunque trabajen como parte de una misma solución.

A continuación, la arquitectura se describe a través de los diagramas C4 elaborados para el proyecto.

Diagrama de Alto Nivel (HLD)

Diagrama de Contexto del Sistema C4

El diagrama de contexto presenta la visión más general del sistema. Aquí no interesa todavía cómo está construido internamente, sino quién interactúa con él y desde qué papel lo hace. En este nivel se identifican los actores principales del negocio: Administrador, Maitre, Mesero, Cocinero y Cliente. Los cuatro primeros corresponden a los roles operativos e internos del sistema, mientras que el cliente se considera como un actor de negocio adicional que, aunque no necesariamente interactúe de manera directa con la plataforma, sí da sentido a varios procesos fundamentales, como las reservaciones y los pedidos.

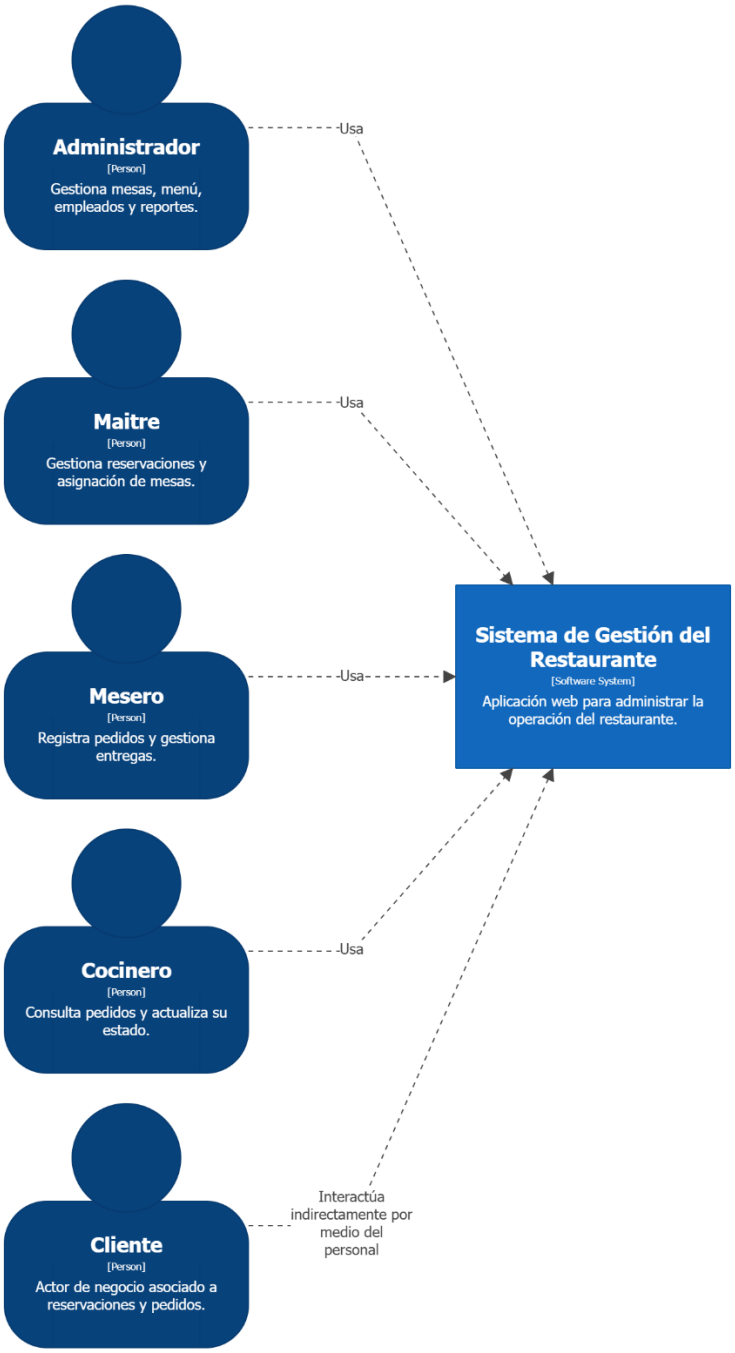


Ilustración 1. Diagrama de Contexto C4.

Visto así, el sistema funciona como un punto central que articula la operación del restaurante. El administrador lo utiliza para gestionar mesas, menú, empleados y reportes; el maitre para registrar reservaciones y asignar mesas; el mesero para tomar pedidos y gestionar entregas; y el cocinero para consultar pedidos y actualizar su estado. El cliente, por su parte, participa de manera indirecta a través del personal del restaurante, lo cual resulta coherente con el enunciado base, donde sus

acciones no se plantean como operaciones autónomas dentro del sistema sino como solicitudes mediadas por otros roles.

Diagrama de Contenedores C4

El diagrama de contenedores baja un nivel y muestra cómo se divide internamente el sistema en bloques tecnológicos principales. En este caso, la solución se estructuró en tres contenedores: Aplicación Web, Backend API y Base de Datos. Es, en el fondo, una organización clásica de tres partes, pero sigue siendo una muy buena elección para este tipo de problema, precisamente porque ordena la responsabilidad de cada parte sin complicar de más el desarrollo.

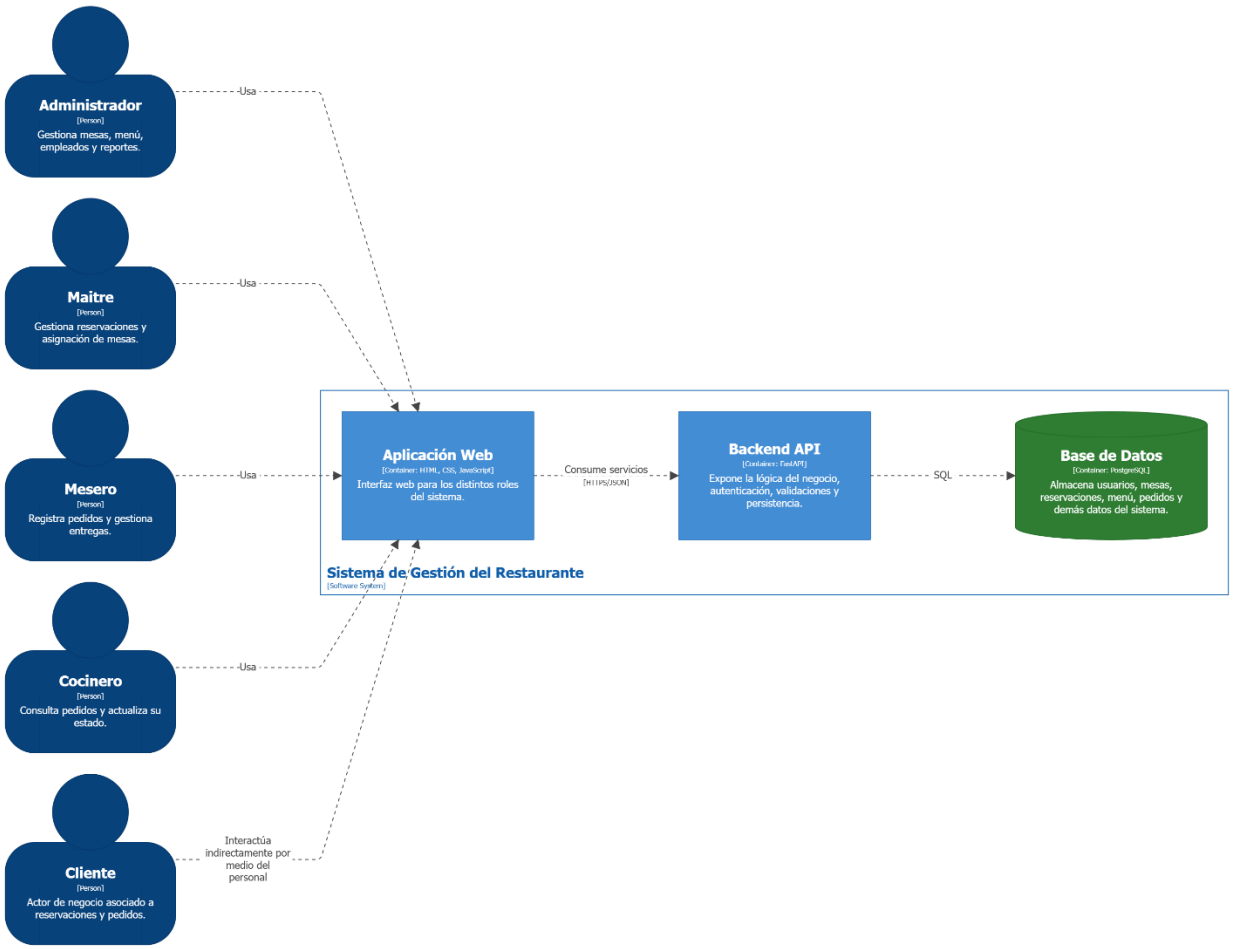


Ilustración 2. Diagrama de Contenedores C4.

La Aplicación Web actúa como la capa de presentación. Es la interfaz con la que interactúan los usuarios del sistema y desde donde se realizan operaciones como iniciar sesión, consultar mesas, registrar reservaciones, administrar el menú, ingresar pedidos o revisar reportes. Su papel no es procesar directamente la lógica del negocio, sino facilitar la interacción y enviar solicitudes al backend.

El Backend API, implementado con FastAPI, concentra la lógica principal del sistema. Allí se ubican los procesos de autenticación, autorización por roles, validaciones de negocio, gestión de estados, notificaciones y coordinación de acceso a los datos. Dicho de otra manera, aquí es donde realmente "piensa" el sistema. Esta elección resulta coherente con la naturaleza de FastAPI como framework orientado al desarrollo de APIs y servicios backend.

Finalmente, la Base de Datos, planteada en PostgreSQL, se encarga de almacenar la información del sistema: usuarios, clientes, mesas, reservaciones, ítems del menú, pedidos, entregas y datos necesarios para reportes y estadísticas. Su ubicación como contenedor separado permite representar de forma clara la persistencia del sistema y refuerza la idea de una arquitectura organizada por responsabilidades.

Diagrama de Componentes de la Aplicación Web C4

El diagrama de componentes de la aplicación web permite observar cómo se organiza internamente la capa de presentación. En lugar de modelarla como una única interfaz genérica, se propuso dividirla en varios componentes que representan las principales vistas o módulos con los que trabajan los usuarios: inicio de sesión, navegación por rol, gestión de usuarios, mesas, reservaciones, menú, pedidos, entregas, reportes y centro de notificaciones.

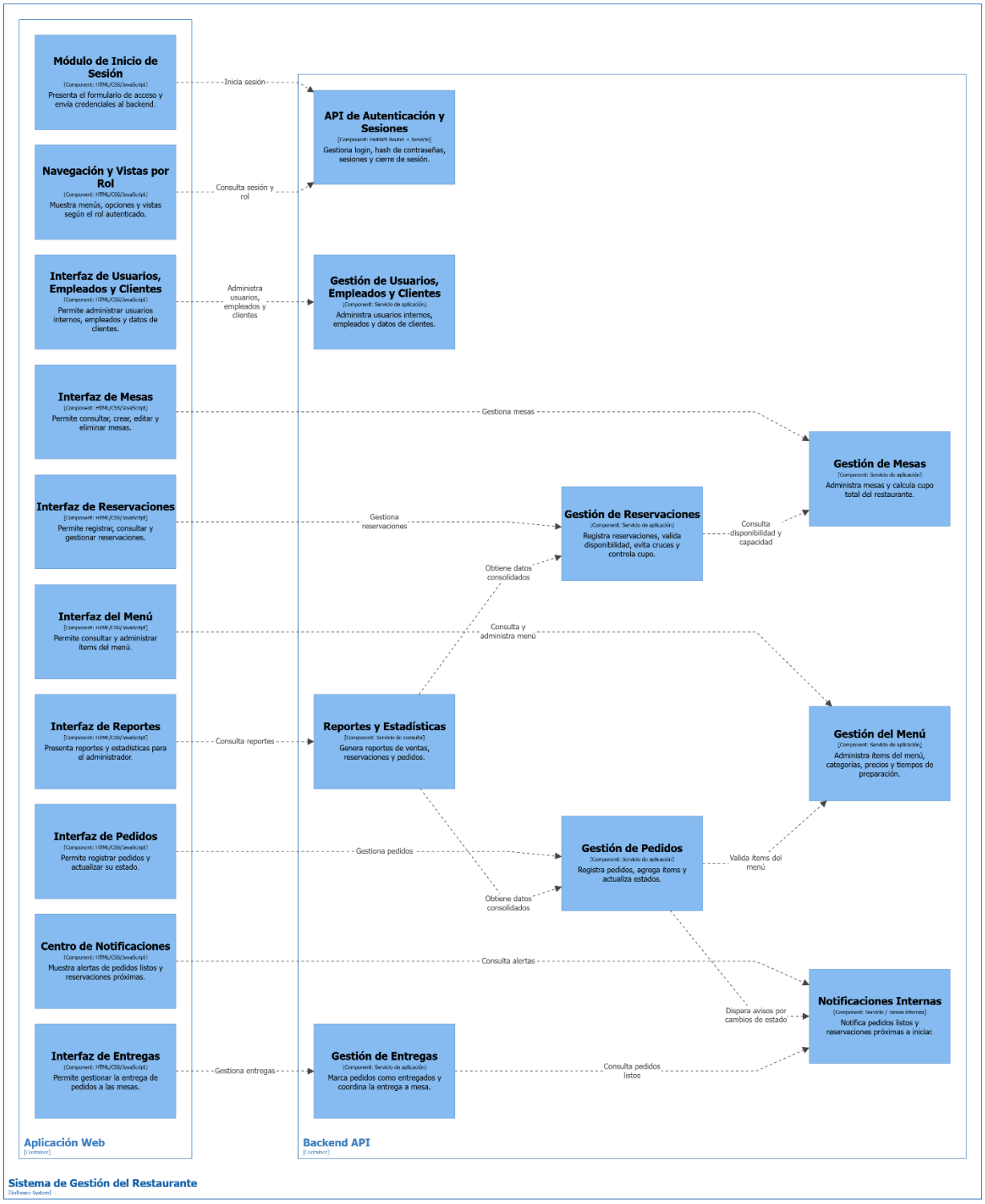


Ilustración 3. Diagrama de Componentes de la Aplicación Web C4.

Esta separación responde a una necesidad práctica, que el sistema debe muestre funcionalidades diferentes según el rol autenticado, y además debe ofrecer flujos claros para cada caso de uso. Por eso, por ejemplo, el componente de inicio de sesión se comunica con la API de autenticación; la

navegación por rol consulta la sesión activa y las capacidades del usuario; y cada interfaz específica se conecta con los servicios backend correspondientes. Así, la aplicación web no mezcla responsabilidades, sino que funciona como una capa organizada en pantallas y módulos bien delimitados.

También resulta importante notar que esta capa no implementa las reglas del negocio por sí sola. Su responsabilidad principal es capturar entradas, mostrar resultados y canalizar las operaciones hacia el backend. Si un mesero registra un pedido, si un maitre consulta una reservación o si un administrador revisa reportes, la web solo presenta la interacción; la validación real ocurre en el backend. Y eso es positivo, porque evita duplicar lógica y mantiene una separación razonable entre presentación y procesamiento.

Otra ventaja de esta descomposición es que hace más evidente cómo la interfaz responde a los requisitos del proyecto. El sistema presenta una estructura funcional alineada con los casos de uso y con las necesidades de cada rol. En ese sentido, la aplicación web se convierte en una proyección directa de la operación del restaurante, es decir, cada módulo refleja una parte concreta del trabajo diario, desde recibir reservaciones hasta monitorear alertas o generar estadísticas.

Diagrama de Componentes del Backend API C4

El diagrama de componentes del backend muestra el núcleo técnico de la solución. Aquí es donde la arquitectura deja de ser solo una distribución general de capas y empieza a verse como un sistema con módulos especializados que colaboran entre sí. En este caso, el backend fue organizado alrededor de componentes que representan tanto servicios funcionales del negocio como mecanismos transversales del sistema.

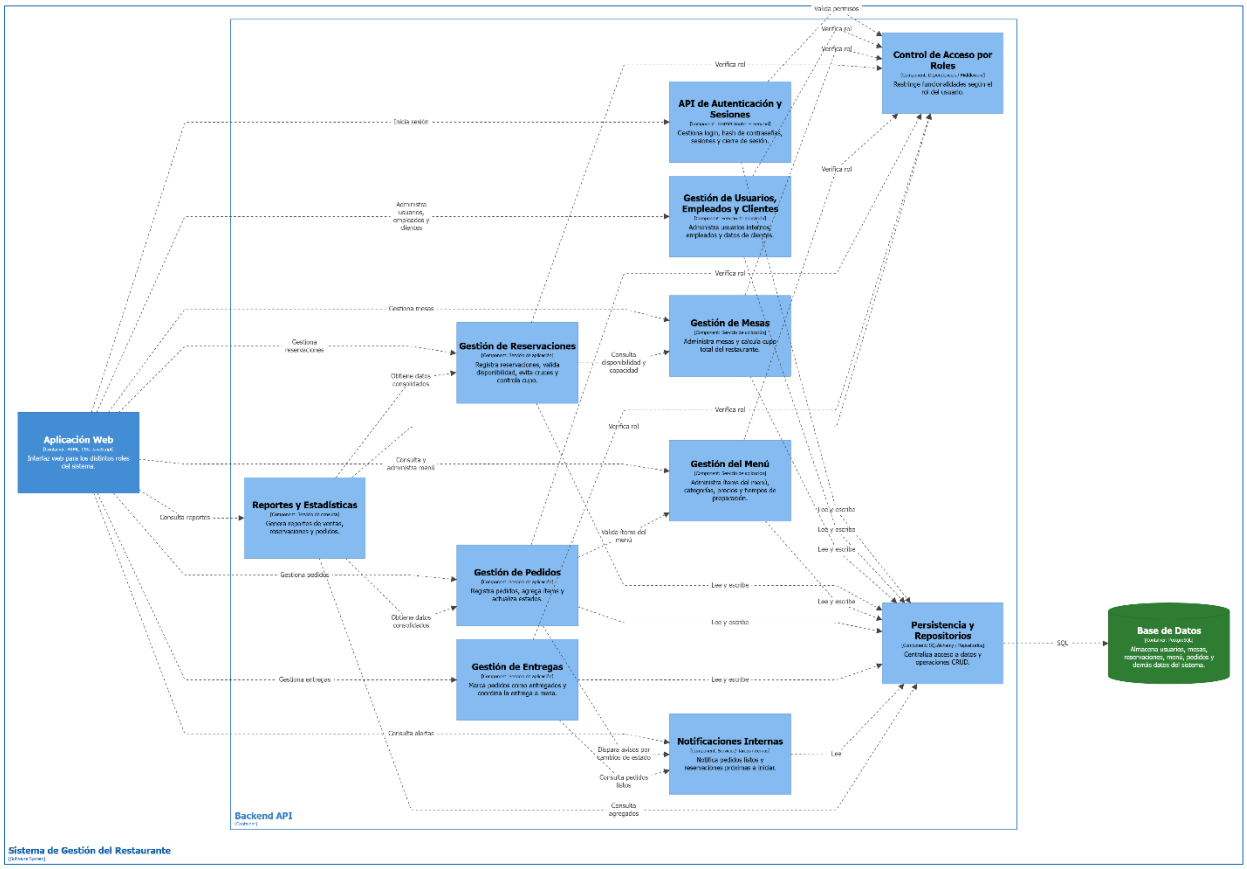


Ilustración 4. Diagrama de Componentes del Backend API C4.

En primer lugar, aparecen los componentes de autenticación y sesiones y control de acceso por roles, que son esenciales porque el proyecto exige login, manejo de sesiones y restricción de funcionalidades según el tipo de usuario. Estos módulos permiten que cada operación se ejecute dentro de un marco de seguridad y autorización definido. No tendría mucho sentido, por ejemplo, que un cocinero pudiera administrar empleados o que un mesero generara reportes globales; por eso el control de acceso no se maneja como un detalle secundario, sino como una pieza estructural del backend.

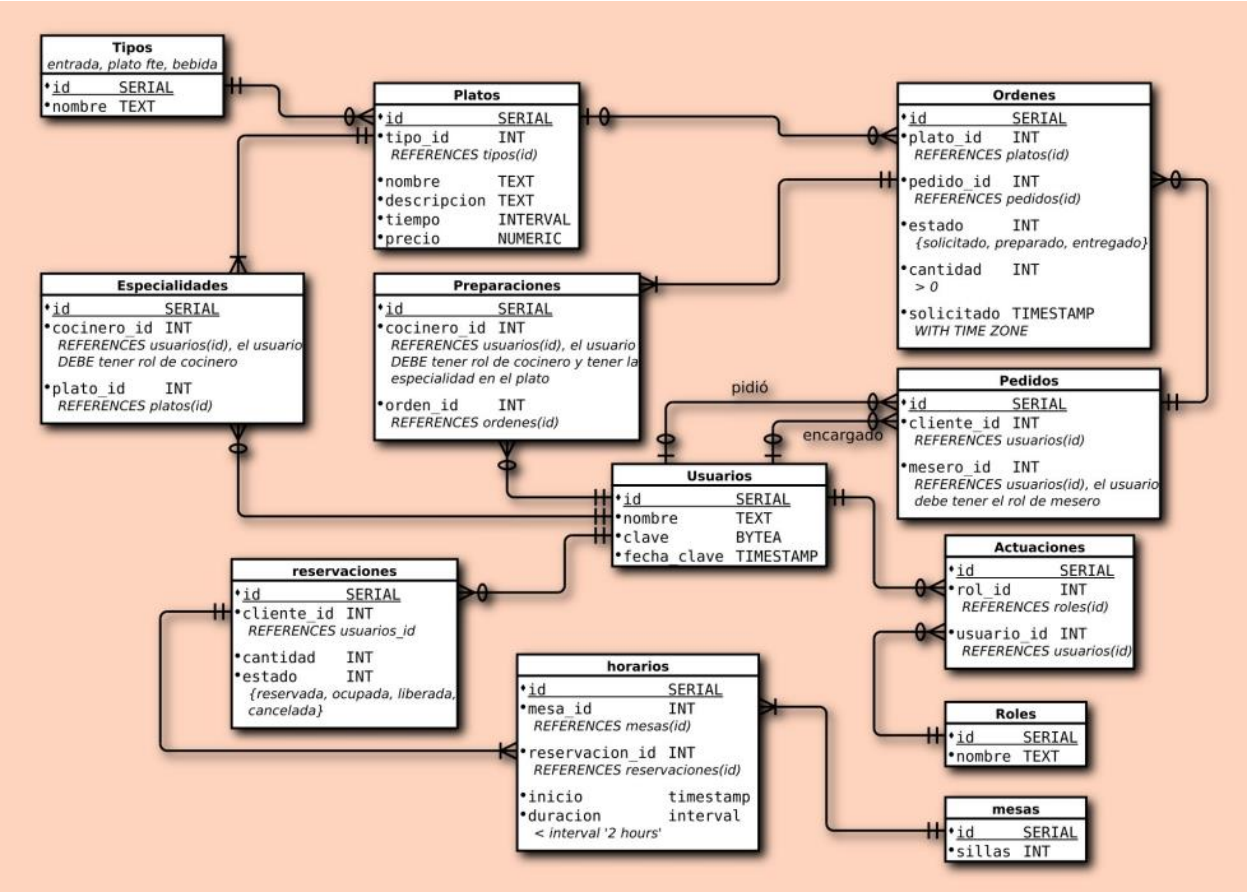
Luego están los servicios orientados al dominio del restaurante: gestión de usuarios, empleados y clientes, gestión de mesas, gestión de reservaciones, gestión del menú, gestión de pedidos, gestión de entregas, notificaciones internas y reportes y estadísticas. Cada uno agrupa responsabilidades concretas. La gestión de mesas administra la capacidad y el cupo total del restaurante; la de reservaciones verifica disponibilidad, evita cruces horarios y controla la ocupación; la de pedidos registra órdenes, agrega ítems y actualiza estados; la de entregas coordina el paso final hacia la mesa; y el componente de reportes consolida la información necesaria para consultas administrativas.

Hay, además, relaciones importantes entre estos componentes. La gestión de reservaciones depende de la información de mesas para conocer disponibilidad y capacidad; la gestión de pedidos

necesita validar ítems del menú; los cambios de estado de un pedido pueden disparar notificaciones internas; y el módulo de reportes obtiene datos consolidados de reservaciones y pedidos. Esa red de interacciones deja ver algo importante: la arquitectura no fue pensada como un conjunto de bloques aislados, sino como una organización modular donde cada servicio conserva su responsabilidad, pero colabora con otros cuando el proceso del negocio lo requiere.

Finalmente, el componente de persistencia y repositorios centraliza el acceso a la base de datos. Esta decisión es relevante porque evita que cada servicio se conecte de manera desordenada a las tablas o mezcle lógica de negocio con lógica de acceso a datos. En cambio, se plantea un punto común de persistencia que mantiene más limpio el diseño y facilita la mantenibilidad del sistema. Desde allí se realizan las operaciones de lectura y escritura necesarias sobre PostgreSQL, respetando la estructura general de la solución.

Modelo de Datos



1. Entidades Principales y Relaciones

El diseño se basa en un esquema relacional normalizado que garantiza la integridad de la información. Las entidades se agrupan en tres núcleos lógicos:

- **Núcleo de Seguridad y Acceso:** Compuesto por Usuarios, Roles y Actuaciones. Implementa la lógica de permisos donde un usuario puede desempeñar distintos papeles operativos.
- **Núcleo de Infraestructura y Clientes:** Incluye Mesas, Reservaciones y Horarios. Aquí se controla el aforo y la restricción de tiempo de 2 horas por servicio.
- **Núcleo Transaccional (Pedidos y Cocina):** Formado por Platos, Tipos, Ordenes y Preparaciones. Este núcleo permite el seguimiento desde que un mesero registra un pedido hasta que el cocinero lo marca como listo.

2. Implementación del ORM (Mapeo Objeto-Relacional)

Para mantener la coherencia con la arquitectura modular propuesta, se definieron clases en Python que representan cada tabla de la base de datos. Esto permite:

- **Abstracción de consultas:** Realizar inserciones y búsquedas utilizando sintaxis de Python en lugar de SQL puro en la capa de servicios.
- **Tipado fuerte:** Aprovechar las ventajas de Python para detectar errores de tipos antes de que los datos lleguen a la base de datos.
- **Gestión de relaciones:** Definir relaciones de "uno a muchos" (como un Pedido con muchas Órdenes) de forma declarativa, facilitando la recuperación de información relacionada.

3. Esquemas de Validación (Pydantic Models)

Dado que FastAPI se orienta a la construcción de APIs, el modelo de datos se extiende hacia los **Schemas de Pydantic**. Estos modelos no interactúan con la base de datos, sino con las peticiones del cliente (JSON):

- **Validación de entrada:** Asegurar que, por ejemplo, el campo cantidad en un pedido sea siempre un número entero positivo.
- **Serialización de salida:** Filtrar qué información se envía al usuario (por ejemplo, nunca enviar el hash de la clave en una consulta de usuario).

4. Estrategia de Persistencia: PostgreSQL

Se seleccionó PostgreSQL como motor de base de datos debido a su robustez en el manejo de transacciones concurrentes.

- **Tablespaces:** Se configuraron espacios de almacenamiento diferenciados para separar los datos operativos de los índices, optimizando el rendimiento de lectura.
- **Particionamiento:** La tabla de Ordenes se implementó con particionamiento por rangos de fecha (TIMESTAMP), permitiendo que el sistema mantenga su agilidad incluso cuando el histórico de ventas crezca significativamente.

Funcionalidades

Funcionalidad Obligatoria: Autenticación y Control de Acceso (RBAC)

Es el componente transversal de seguridad. Utiliza el estándar **OAuth2** con flujo de contraseña y **JSON Web Tokens (JWT)** para la persistencia de la sesión.

- **Lógica:** Al ingresar al sistema, el backend valida las credenciales contra la tabla de usuarios. Si son correctas, genera un token que contiene los roles del usuario.
- **Control por Roles:** Se implementaron "dependencias" en FastAPI que actúan como guardianes en cada ruta. Por ejemplo, solo los usuarios con el rol `Cocinero` pueden visualizar la cola de preparación, mientras que solo el `Administrador` tiene acceso a la creación de usuarios y reportes.

Casos de Uso Principales

1. **Gestión de Mesas:** Permite el control físico del restaurante. El sistema gestiona el ciclo de vida de una mesa: Disponible, Reservada u Ocupada. Al ser un sistema centralizado, cualquier cambio realizado por el Maitre es visible instantáneamente para los meseros, evitando conflictos de asignación.
2. **Registro de Pedidos:** Representa el flujo transaccional más dinámico del sistema.
    a. **Proceso:** Un mesero selecciona una mesa activa y registra los ítems del menú.
    b. **Implementación:** FastAPI procesa estas peticiones de forma asíncrona, permitiendo que múltiples órdenes se registren simultáneamente. Cada pedido se vincula a una mesa y a un cliente, disparando automáticamente la notificación en el módulo de cocina.
3. **Reportes y Estadísticas:** Módulo de inteligencia de negocio exclusivo para el Administrador.

    a. **Funcionalidad:** Realiza cálculos agregados sobre el histórico de pedidos para mostrar ingresos totales y frecuencia de consumo.
    b. **Optimización:** Gracias al particionamiento de la base de datos por fechas, las consultas de reportes mensuales mantienen un alto rendimiento sin importar el volumen de datos históricos.

Funcionalidades Adicionales

1. **Gestión de Empleados:** Un módulo CRUD completo que permite administrar el talento humano del restaurante. No solo gestiona los datos personales, sino que vincula a cada empleado con sus especialidades técnicas (importante para los cocineros) y sus credenciales de acceso al sistema.
2. **Historial de Clientes:** Permite realizar un seguimiento de los consumos previos de cada comensal. Al consultar un ID de cliente, el sistema recupera todas las órdenes asociadas, lo que facilita la personalización del servicio y la identificación de clientes frecuentes.
3. **Validación de Cupo:** Es una lógica de control de integridad que se ejecuta antes de confirmar cualquier reserva o asignación de mesa.
    a. **Lógica:** El backend compara el número de personas solicitadas contra el campo `capacidad` de la mesa en la base de datos. Si el grupo excede el cupo, el sistema impide el registro y sugiere buscar una mesa con mayor capacidad, garantizando la comodidad y seguridad dentro del establecimiento.

Evidencias de funcionamiento